

8086 CPU 有两种工作模式:

- 最小模式: 系统中只有8086一个微处理器, 直接产生所有总线控制信号, 其他外加总线逻辑控制部件最少
- 最大模式: 8086作主处理器, 其他作协处理器(辅助处理器)
 - 8087: 数值运算(提升速度)
 - 8089: 输入/输出操作(减少占用时间)

* 或 - 表示低电平有效

8086 CPU 的引脚 $MN/\overline{MX}$ 端接高电平构成最小模式, $MN/\overline{MX}$ 端接低电平构成最大模式

8086 有40个引脚, 采用“双列直插式”的封装形式, 为减少芯片上的引脚数目, 分时复用地址/数据总线

三态: 高电平、低电平、高阻态

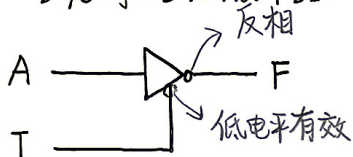
三态门和以 D 触发器形成的锁存器是微机接口电路中最常使用的两类逻辑电路

- 功率放大
- 信号保持
- 导通开关
- 导通开关

器件共用总线时, 一般使用三态电路

需要使用总线时打开三态门, 不使用时关闭使之处于高阻

三态门(三态缓冲器)



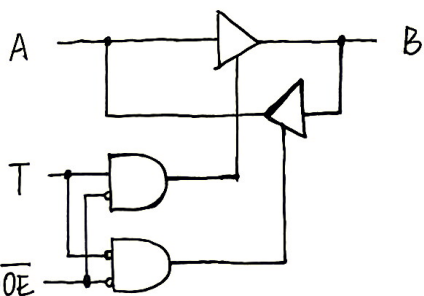
表示反相或低电平有效

三态门具有单向导通和三态的特性

T 为低电平时: 输出为高阻抗(三态)

T 为高电平时: 输出为输入的反相

双向三态门(双向三态缓冲器)



$\overline{OE}=0$, 导通 $\overline{OE}=1$, 不导通

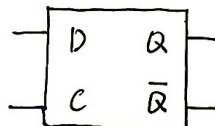
$\begin{cases} T=1 & A \rightarrow B \\ T=0 & A \leftarrow B \end{cases}$

三态输出:

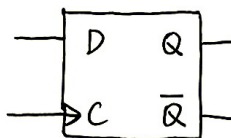
输出控制信号有效时, 允许数据输出;

无效时, 不允许数据输出, 呈高阻态.

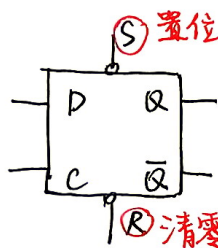
D 触发器



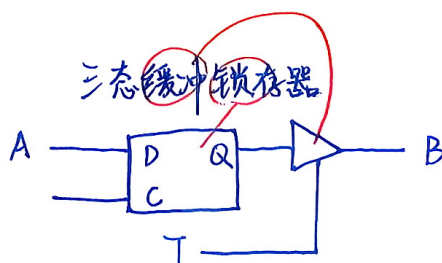
电平锁存: 高电平通过
低电平锁存



上升沿锁存(通常用负脉冲触发)



带有异步清零的电平控制的
锁存器



8086 最小模式下引脚信号说明 (最大模式下许多总线控制信号不由 8086 直接产生而是通过总线控制器 8288 产生, 最小模式下提供的总线控制信号的引脚就要重新定义)

① $\overline{AD}_{15} \sim \overline{AD}_0$ (地址/数据复用引脚) 输出, 三态

传送地址或数据的分时复用引脚, 传送地址时三态输出, 传送数据时三态双向输入/输出

- 在总线周期 T_1 状态, CPU 在这些引脚上输出存储器或 I/O 端口的低 16 位 ($A_{15} \sim A_0$) 地址
- 在总线周期 $T_2 \sim T_4$ 状态, 用来传送数据 ($D_{15} \sim D_0$)

在中断响应及系统总线“保持响应”周期, $\overline{AD}_{15} \sim \overline{AD}_0$ 置高阻态

② $A_{19} \sim A_{16} / S_6 \sim S_3$ (地址/状态复用引脚) 输出, 三态

- 在总线周期 T_1 状态, 用来输出 20 位地址信息的最高 4 位 ($A_{19} \sim A_{16}$)

当 CPU 访问 I/O 端口 (外设端口) 时, $A_{19} \sim A_{16}$ 是 “0”

- 在总线周期 $T_2 \sim T_4$ 状态, 用来作状态线使用

$S_6 \sim S_3$ 输出状态信息

S_6 保持 0, 说明 8086 连在总线上	$\left\{ \begin{array}{l} \text{置 1, 允许屏蔽中断请求} \\ \text{置 0, 禁止屏蔽中断} \end{array} \right.$
S_5 指示中断允许标志 IF 的状态	
S_4, S_3 指示使用何段寄存器	

S_4	S_3	当前使用
0	0	ES
0	1	SS $\rightarrow$ 对存储器寻址
1	0	CS $\rightarrow$ 对 I/O 端口或中断向量寻址
1	1	DS

当系统总线处于“保持响应”状态, 这些引脚被置成高阻态

③ $\overline{BHE} / S_7$ (高 8 位数据总线允许/状态复用信号) 输出, 三态

- 在总线周期 T_1 状态, 8086 在 $\overline{BHE} / S_7$ 输出低电平有效信号, 表示高 8 位数据总线 $D_{15} \sim D_8$ 上数据有效

若 $\overline{BHE} = 1$, 则表示仅在 $D_7 \sim D_0$ 上传送 8 位数据

$\overline{BHE}$ 与 A_0 信号配合, 指出当前传送的数据在总线上将以何种格式出现, 应在存储器的奇/偶体的存储单元进行字节/字的读/写操作

$\overline{BHE}$ 也作对 I/O 电路或中断响应时的片选条件信号

- 在总线周期 $T_2 \sim T_4$ 状态, S_7 输出状态信息, 在“保持响应”周期被置成高阻态

④ $MN/\overline{MX}$ (组态选择) (最小/最大模式控制信号输入端)

$MN/\overline{MX}$ 接 +5V 时, CPU 工作在最小模式; 接地时, 工作在最大模式

⑤ $\overline{RD}$ (读信号) 输出, 三态, 低电平有效

当 $\overline{RD} = 0$, CPU 读存储器或 I/O 端口 (由 $M/\overline{IO}$ 信号区分)

在读总线周期的 T_2, T_3, T_w 状态, $\overline{RD} = 0$

在“保持响应”周期置成高阻态

⑥ $\overline{WR}$ (写信号) 输出, 三态, 低电平有效

当 $\overline{WR} = 0$, CPU 写存储器或 I/O 端口

在读总线周期的 T_2, T_3, T_w 状态, $\overline{WR} = 0$

在 DMA 方式时置成高阻态

- ⑦ M/I $\overline{O}$ (存储器/输入输出接口操作选择控制信号) 输出, 三态
引脚输出高电平时, 地址总线 $A_{19} \sim A_0$ 提供 20 位 M 地址; 低电平时, $A_{15} \sim A_0$ 提供 16 位 I/O 口地址
在前一总线周期的 T_4 状态, M/I $\overline{O}$ 有效, 直至本周期 T_4 状态为止

在 DMA 方式时, M/I $\overline{O}$ 被置为高阻态

$\overline{RD}$	$\overline{WR}$	M/I $\overline{O}$	操作
0	1	0	读 I/O
1	0	0	写 I/O
0	1	1	读 M
1	0	1	写 M

- ⑧ ALE (地址锁存允许信号输出) 输出, 三态, 高电平有效

8086 的地址/数据总线分时复用, CPU 与内存, I/O 电路交换信息时, 先利用该总线传地址信息, 再传数据信息

ALE 高电平有效时表示 $AD_{15} \sim AD_0$ 和 $A_{19} | S_6 \sim A_{16} | S_3$ 正在传送地址信息

为将地址信息保存, ALE 将地址信息锁存到地址锁存器 (8282/8283) 的锁存信号
在任一总线周期的 T_1 状态产生正脉冲, 利用其下降沿来锁存地址信息, 实现地址信息和数据信息分时复用传送

- ⑨ $\overline{DEN}$ (数据允许信号) 输出, 三态, 低电平有效

$\overline{DEN}$ 用作数据收发器 8286/8287 的输出允许信号

当 CPU 访问存储器或 I/O 端口的总线周期的后一段时间内和中断响应周期中, 此信号低电平有效
在 DMA 方式时被置成高阻态

- ⑩ DT/ $\overline{R}$ (数据发送/接收控制信号) 输出, 三态

DT/ $\overline{R}$ 用来控制数据收发器 8286/8287 的数据传送方向

DT/ $\overline{R}$ = 1 时, CPU 发送数据, 完成写操作

DT/ $\overline{R}$ = 0 时, CPU 接收数据, 完成读操作

在 DMA 方式时被置成高阻态

- ⑪ READY (准备就绪信号) 输入, 高电平有效

由存储器或 I/O 端口发来的响应信号表明外设已准备好进行数据传送,
CPU 在每个总线周期的 T_3 状态检测 READY 信号线, 若是低电平则要在 T_3 状态结束后 CPU 插入一个或几个 T_w 等待状态, 直至 READY 信号有效才进入 T_4 状态, 完成数据传送

- ⑫ RESET (复位输入信号) 高电平有效

CPU 接收到该信号后, 停止当前操作并初始化寄存器 PS、ES、SS, 标志寄存器 PSW, 指令指针 IP, 指令队列而使 $CS = FFFFH$. RESET 信号至少保持 4 个时钟周期以上的高电平, 当变为低电平时 CPU 重启, 8086 将从地址 $FFFF0H$ 开始执行指令
8086 复位后 $CS = FFFFH$, $IP = 0000H$, 程序入口在物理地址 $FFFF0H$
(通常在 $FFFF0H$ 单元开始的几个单元中存放一条无条件转移指令 (JMP), 将入口转到引导和装配程序中, 实现系统初始化, 引导监控程序或系统程序)

⑬ INTR (可屏蔽中断请求信号的输入端) 输入、高电平有效

主机与外设进行数据交换通常采用可屏蔽中断。

CPU在每条指令的最后一个时钟周期检测该引脚输入信号

一旦检测到该信号有效并且中断允许标志位IF=1时, CPU在当前指令执行完成后转入中断响应周期, 读取外设接口的中断类型码, 然后在寄存器中断向量表中找中断服务程序的入口地址, 转入执行中断服务程序

该中断请求是否响应受控于IF(中断允许标志), 可被屏蔽

⑭ INTA (中断响应信号) 输出、低电平有效

是CPU对外部发来的中断请求信号INTR的响应信号,

该信号低电平有效时, 告知外设其中断请求已得到CPU允许, 外设接口可以向数据总线上放置中断类型码, 以便取得相应中断服务程序的入口地址

⑮ NMI (非屏蔽中断请求信号输入端) 低电平到高电平上升沿触发

优先级高于INTR, 不受中断允许标志位IF的影响, 不能用软件屏蔽。

NMI引脚一旦接收到上升沿信号, CPU就在当前指令执行结束后立即响应中断, 进入相应中断处理程序。

经常处理电源掉电等紧急情况

主机与外设进行数据交换通常采用可屏蔽中断

⑯ TEST (测试输入信号) 低电平有效

测试信号在CPU执行WAIT指令期间, CPU每隔5个时钟周期对TEST引脚测试一次,

若测试到TEST为高电平, CPU处于空转等待状态;

若测试到TEST为低电平, CPU继续执行下一条指令。

TEST引脚信号用于多处理器系统中, 实现8086与协处理器间的同步协调

⑰ HOLD (总线保持请求信号) 输入, 高电平有效

在最小模式系统中, 其他部件要求占用总线时, 可通过对此引脚施加高电平表示向CPU请求使用总线

若CPU允许让出总线控制权, 则在当前总线操作周期完成后于T₄状态在HLDA引脚送出高电平回答信号, 作为对刚才总线请求的响应

同时, CPU让地址/数据总线和控制状态线处于悬空状态, CPU放弃对总线的控制权,

而申请总线请求的部件收到HLDA信号后获得总线控制权 从有效回到无效表示使用结束, 通知CPU收回对总线的控制权。

⑱ HLDA (总线保持响应信号) 输出, 高电平有效

表示CPU已响应总线请求且已释放总线

若CPU检测到HOLD信号有效且CPU允许让出总线, 在当前总线周期结束时,

在T₄状态发出HLDA信号, 响应总线请求, 立即让出总线使用权, 置三总线为高阻态。

获得总线控制权的部件在使用总线完毕后, 使HOLD低电平无效, 具有三态输出能力的CB

CPU才将HLDA置低电平, CPU重获三总线使用权

AB

DB

具有三态输出能力的CB

⑲ CLK (时钟输入端) 系统通过该引脚给CPU提供内部定时信号

时钟信号由8284时钟发生器产生, 8086 CPU使用的时钟频率因芯片型号不同而不同

8086: 5MHz 8086-1: 10MHz 8086-2: 8MHz

⑳ V_{CC} (+5V), GND (地)

CPU所需电源V_{CC}=+5V; GND为地(有2条引脚: 1和20)

CPU引脚如何与外部连接?

(8086 CPU 最小模式系统配置, 总线形成)

20位地址总线: 采用3个三态透明锁存器 8282/8283 进行锁存/驱动

8位数据总线: 采用2个总线收发器 8286/8287 进行驱动
数据收发器

系统控制信号: 由CPU引脚直接提供

注: 最小模式系统中, 有 8086 CPU、存储器、I/O 接口芯片、

3片 8282/8283 (或 74LS 373) 地址锁存器、2片 8286/8287 (或 74LS 245) 总线收发器、

1片 8284A 时钟发生器 (提供 $\left\{ \begin{array}{l} \text{时钟信号 CLK} \\ \text{经过同步的就绪信号 READY} \\ \text{系统复位信号 RESET} \end{array} \right.$)

8088 CPU 和 8086 CPU 的区别?

8088 CPU 的内部数据总线宽度是 16 位, 外部数据总线宽度是 8 位, 8088 CPU 又被称为 16 位微处理器
8086: 16
8088: 8

内部结构方面:

8088

① 8088 的指令队列只有 4 个字节, 出现 1 个空字节时 BIU 就会自动从内存单元取指令来补充指令队列

② 8088 中 BIU 的总线控制电路与外部交换数据的总线宽度是 8 位而 EU 的内部总线是 16 位, 对 16 位数的存储器读/写操作必须 2 个总线周期才可完成。

引脚功能方面:

① 8088 的外部数据总线仅有 8 条, 仅能传 8 位数据, 只有 8 个复用的地址/数据总线 $AD_7 \sim AD_0$, 只要 1 片总线收发器 8286; 8086 的 $AD_{15} \sim AD_8$ 变为 $A_{15} \sim A_8$, 传输地址信息

② 8088 的最小模式系统读/写周期中, 8086 的 $\overline{M/\overline{IO}}$ 信号变为 $IO/\overline{M}$ 信号
 $\overline{BHE}$ 改为 SS_0 信号, 与其它信号合作完成总线周期操作。

CPU 引脚如何相互配合, 实现总线操作, 控制系统工作?

(8086 总线时序)

8086 CPU 的操作是在时钟脉冲 CLK 的统一控制下进行的, 按规定的节拍执行指令。

CPU 初始化操作 (上电复位操作):

CPU 上电以后自动将其内部各器件置成预先已知所要求的内容和状态的操作

8086 CPU 的上电复位操作通过在 RESET 引脚上施加一定宽度的正脉冲信号来实现, 初始化后微处理器才可以在时钟信号 CLK 控制下按规定节拍工作。

8088 总线时序

时序是指信号高低电平(有效或无效)变化及相互间的时间顺序关系

- CPU 时序决定了系统各部件间的同步和定时
- 总线时序描述了 CPU 引脚如何实现总线操作

总线操作

CPU 通过总线对外的各种操作

例如:

- CPU 读/写
- I/O 读/写
- 中断响应
- 总线请求及响应

CPU 正进行内部操作而不进行实际对外操作的空闲状态 T_1

描述总线操作的微处理器时序有 3 级: 指令周期 $\rightarrow$ 总线周期 $\rightarrow$ 时钟周期

① 指令周期:

一条指令经取指、译码、读写操作数到执行完成的过程

若干总线周期组成一个指令周期

② 总线周期:

CPU 通过总线操作与外部(存储器或 I/O 端口)进行一次数据交换的过程

8088 的基本总线周期需要 4 个时钟周期 ($T_1 \sim T_4$)

总线周期中的时间周期又称 "T 状态"

③ 时钟周期:

时钟周期的时间长度是时间频率的倒数

- 当要延长总线周期时, 插入等待状态 T_w
- 当 CPU 进行内部操作而没有对外操作时, 其引脚处于空闲状态 T_1
- 任何指令的取指阶段都需要存储器读总线周期, 读取的内容是指令代码
- 任何一条以存储单元为源操作数的指令都将引起存储器读总线周期。
任何一条以存储单元为目的操作数的指令都将引起存储器写总线周期
- 只有执行 IN 指令才出现 I/O 读总线周期, 只有执行 OUT 指令才出现 I/O 写总线周期
- CPU 响应可屏蔽中断时生成中断响应总线周期

总线操作中如何实现时序同步?

① CPU 总线周期采用同步时序:

CPU

I/O、M

各部件均以系统时钟信号为基准, 快速部件会插入等待状态来等待慢速部件

② CPU 与外设接口采用异步时序:

通过应答联络信号实现同步操作

最小组态的总线时序

4种总线周期: 存储器读总线周期、存储器写总线周期、I/O读总线周期、I/O写总线周期

T_w 同步时序通过插入
等待状态来使速度差别
大的两部分保持同步

在读写总线周期中,

判断是否插入 T_w :

在 T_3 前沿检测 $READY$
(M或I/O已准备好数据
传送) 引脚是否有效

若 $READY$ 有效,

则执行完该 T 状态

进入 T_4 状态;

若 $READY$ 无效,

则在 T_3 、 T_4 间插入一个

T_w (与 T_3 等效),

随后再转到 T_3

前沿检测

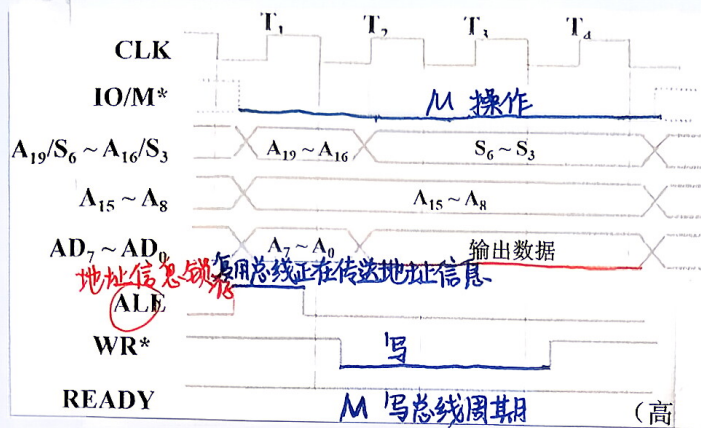
何为分时复用?

一个引脚在不同时刻
具有两个甚至多个作用.

• 目的是减少对外引脚
的个数

• 常见的总线复用是

数据 and 地址引脚复用
8086/8088 的数据地址
线就采用了总线复用的
方法

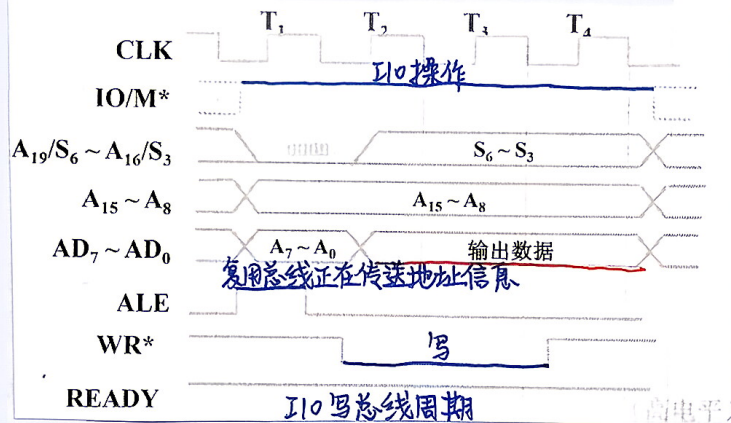


T_1 状态: 输出20位存储器
地址 $A_{19} \sim A_0$

T_2 状态: 输出控制信号 $\overline{WR}$
和数据 $D_{15} \sim D_0$

T_3 、 T_w : 检测数据传送
能否完成

T_4 : 完成数据传送

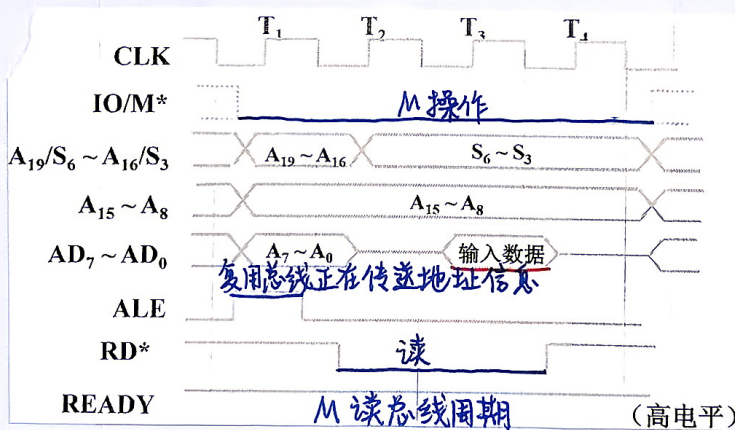


T_1 状态: 输出16位 I/O
地址 $A_{15} \sim A_0$

T_2 状态: 输出控制信号 $\overline{WR}$
和数据 $D_{15} \sim D_0$

T_3 、 T_w : 检测数据传送
能否完成

T_4 : 完成数据传送

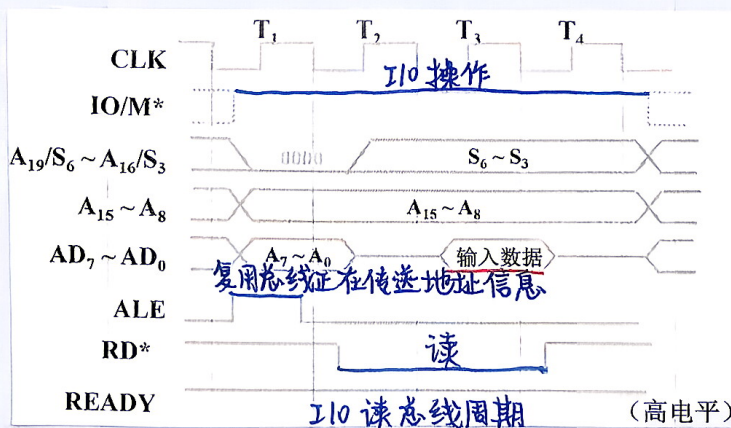


T_1 状态: 输出20位存储器
地址 $A_{19} \sim A_0$

T_2 状态: 输出控制信号 $\overline{RD}$

T_3 、 T_w : 检测数据传送
能否完成

T_4 : 前沿读取数据,
完成数据传送



T_1 状态: 输出16位 I/O
地址 $A_{15} \sim A_0$

T_2 状态: 输出控制信号 $\overline{RD}$

T_3 、 T_w : 检测数据传送
能否完成

T_4 : 前沿读取数据,
完成数据传送

8086/8088 的存储器组织

存储器的分段与地址形成

存储单元地址：为标识和存取每一个存储单元而给每一个存储单元规定的编号

存储单元内容：一个存储单元中存放的信息

- 存储器以 1 字节 = 8 bit 为编程单位，^{16位。}一个字要占用相继的 2 个字节
- 每个字节单元都有唯一的地址编码
- 地址用无符号整数来表示（编程用 16 进制表示）
- 低位字节存入低地址，高位字节存入高地址，字单元地址用其低地址来表示
- 机器以偶地址访问（读写）存储器

寄存器：微处理器（CPU）内部暂存数据的存储单元，以名称表示（如 AX）

存储器：亦称主存/内存，可直接与 CPU 进行数据交换

外存：用以长久保存数据的外部存储介质，只能通过主存间接地与 CPU 进行数据交换（如硬盘、光盘、磁带、U 盘）

程序及其数据可以长久地存放在外存，在运行需要时才进入主存

二进制位：存储一位二进制数，0 或 1

字节：8 个二进制位 $D_7 \sim D_0$

字：16 个二进制位，2 字节 $D_{15} \sim D_0$

双字：32 个二进制位，4 字节 $D_{31} \sim D_0$

最低有效位 LSB：数据最低位 D_0 位

最高有效位 MSB：数据最高位 字节是 D_7 位、字是 D_{15} 位、双字是 D_{31} 位

存储单元的地址和内容

字节	7	6	5	4	3	2	1	0	
	1	0	0	1	1	1	1	1	0000H
	15	14	13	12	11	10	9	8	0001H
	0	0	0	1	1	1	1	0	0002H
	1	1	0	1	0	1	1	1	0003H
	⋮	⋮	⋮	⋮	⋮	⋮	⋮	⋮	

0000H

(0000H) = 9FH

0001H

(0001H) = 26H

0002H

(0002H) = 1EH

0003H

(0003H) = D7H

(0000H) = 269FH

(0002H) = D71EH

字单元地址用“低地址”表示

存储器

字	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
	0	0	1	0	0	1	1	0	1	0	0	1	1	1	1	1

高位字节

低位字节

12340H	⋮
12341H	12H
12342H	34H
12343H	56H
12344H	78H
12345H	90H
	⋮

- 1个字节的內容是该字节单元内存放的二进制信息

字节

(12340H) = 12H

(12341H) = 34H

- 1个字的内容是该字地址所指单元及其后继单元内容的拼接

字

(12340H) = 3412H

(12341H) = 5634H

- 1个双字的内容是该字地址所指单元及后继3单元内容的拼接

双字

(12340H) = 78563412H

(12341H) = 90785634H

对“字”来说，高存储单元（地址大的存储单元）的值是“字”内容的高8位。

低存储单元（地址小的存储单元）的值是“字”内容的低8位

8086是“小端存储”！

8086/8088 CPU的地址线是20位的,最大可寻址空间为 $2^{20} = 1\text{MB}$

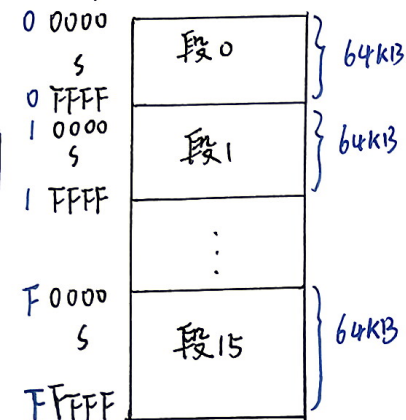
而8086/8088的内部寄存器是16位的,只能寻址 $2^{16} = 64\text{KB}$

为使16位寄存器有效访问1MB存储空间,8086/8088 CPU采用“内存分段”的管理模式:

将1MB存储空间分为 $2^4 = 16$ 个互不重叠的逻辑段,每个逻辑段最大64KB,段内地址可用16位表示

形如:

段地址须为16的倍数



段不能起始于任一地址而必须从任一小段的首地址开始。

从0地址开始,每16字节是1小段。

00000H ~ 0000FH 就是1小段(16个字节!)

00000H、00010H、00020H ... 就是1小段的首地址

对于每一小段的首地址,

由于20位地址的16进制表示中最低位是0(低4位是0),

因此把这个最低位的0省略,→后4位在需要时通过“左移4位”获得即可

用16位表示段起始地址,可保存在16位段寄存器中

其实也可依据实际需要确定段大小,也允许重叠(每个段区大小不一定是64KB),

但各存储单元的内容坚决不可能发生冲突。

各段在存储器中的分配由操作系统完成

段内偏移: 存储单元的实际地址与其所在段的段地址间的距离

亦称“有效地址(EA)”或“偏移量(Offset)”

通过段地址与偏移量

就可唯一确定某一内存单元在存储器内的具体位置

每个存储单元的唯一20位地址

段内相对段起始地址的偏移(16位)

$$\text{物理地址 (PA)} = \text{段地址} \times 16 + \text{偏移量}$$

① 左移4位

② 加

段起始地址的高16位

存储器分段的原则:

段起始地址必须是某一小段的首地址(即低4位是0000B),

而段的大小可以是64KB范围内的任意字节。

物理地址与逻辑地址的关系:

对物理地址来说,当段地址变化时,只要对其偏移量进行相应变化即可对应同一物理地址。

故同一物理地址可有多逻辑地址

8086/8088 CPU 哪些寄存器可用来指示存储器地址?

答: ① 段寄存器 CS、DS、SS、ES

② 指令指针寄存器 IP

③ 通用寄存器 AX、BX、CX、DX

④ 堆栈指针寄存器、基址指针寄存器 SP、BP

⑤ 变址寄存器 SI、DI

这6个可用来指示段内偏移地址

在 DEBUG 程序中, 有一条指令语句 2000:0030 MOV AL,8

段地址、偏移地址、物理地址分别是多少? 存放段地址和偏移地址的寄存器是?

答: ① 段地址: 2000H

偏移地址: 0030H

物理地址: 段地址 $\times 16$ + 偏移地址 = $2000H \times 16 + 0030H = 20030H$

下一条指令在代码段中的

偏移地址

CS:IP 取指令

SS:SP 操作栈顶

DS:EA 或 ES:EA 取数据

数据段 附加段

② 由于这是指令, 存放在代码段. 故存放段地址的寄存器是 CS

存放偏移地址的寄存器是 IP

在汇编语言程序中, 存储单元通常是用逻辑地址标识的而不是用物理地址标识的

存储器的逻辑地址与物理地址

逻辑地址 段地址: 偏移地址		物理地址	
1000 : 0000H	10010011	10000H	} 16字节, 1 "小段"
1000 : 0001H	01001110	10001H	
1000 : 0002H	00001000	10002H	
...	...	...	
1000 : 000DH	11010110	1000DH	
1000 : 000EH	01111110	1000EH	
1000 : 000FH	10010101	1000FH	

字节内容: (10000H) = 93H (10001H) = 4EH

字内容: (10000H) = 4E93H (10001H) = 084EH

1. 8086 CPU 把 1MB 存储器分成若干逻辑段, 段不能起始于任意地址, 必须位于 16 个字节的边界上
每个段最大为 64K 字节, 8086 CPU 中设置了 4 个寄存器用来存放逻辑段的段地址, 它们是
CS、DS、ES、SS。

2. 已知逻辑地址得到物理地址的计算方法是

$$\text{段地址} \overset{\times 10H}{\overset{\times 16}{\text{左移 4 位}}} + \text{偏移地址}$$

3. 1 个指令周期就是 1 个总线周期 (X)

1 个时钟周期亦称 1 个 T 状态 (✓)

1 个基本总线周期有 4 个 T 状态 (✓)

执行 1 条指令所需的时间是 1 个周期 (✓)

4. 8086 CPU 读内存时, 控制信号 $\overset{\Delta}{M^*}/\overset{\Delta}{IO}$ 和 $\overset{\Delta}{DT}/\overset{\Delta}{R^*}$ 是 00。